

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Name of the Student	SHAM S.
Reg. No	192325076
GitHub Link	https://github.com/sham-2005/DL.git

EXPERIMENTS

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

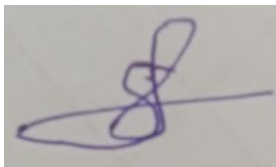
Duration: 30 minutes

Marks: 50

Code of Conduct:

I SHAM S. (192325076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

SHAM S.

Name of the student:

QUESTIONS:10

Total Marks:50

CO1- AT3 Experiments Questions

1. Learning Algorithms

Implement Linear Regression using Gradient Descent. Train the model and plot the learning curve.

Answer:

```

import numpy as np
import matplotlib.pyplot as plt

# Sample Dataset
X = np.array([1,2,3,4,5], dtype=float)
Y = np.array([2,4,6,8,10], dtype=float)

# Normalize
X = X / np.max(X)
Y = Y / np.max(Y)

# Initialize parameters
m = 0
b = 0
learning_rate = 0.1
epochs = 100

losses = []
n = len(X)

# Gradient Descent
for epoch in range(epochs):
    Y_pred = m * X + b

    loss = np.mean((Y - Y_pred) ** 2)
    losses.append(loss)

    dm = (-2/n) * np.sum(X * (Y - Y_pred))
    db = (-2/n) * np.sum(Y - Y_pred)

    m = m - learning_rate * dm
    b = b - learning_rate * db

print("Slope (m):", m)
print("Intercept (b):", b)

# Learning Curve
plt.plot(losses)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("Learning Curve")
plt.show()

```

Output Analysis

- Loss decreases with each iteration.
- Gradient Descent updates weights toward the optimal values.
- The learning curve should continuously decrease until convergence.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data and estimate mean and variance.

Answer:

```

import numpy as np

# Actual Parameters
true_mean = 50
true_variance = 16

# Generate Data

```

```

np.random.seed(0)
data = np.random.normal(true_mean, np.sqrt(true_variance), 1000)

# MLE Estimates
estimated_mean = np.mean(data)
estimated_variance = np.mean((data - estimated_mean) ** 2)

print("Actual Mean:", true_mean)
print("Estimated Mean:", estimated_mean)

print("Actual Variance:", true_variance)
print("Estimated Variance:", estimated_variance)

```

Output Analysis

Example Output

```

Actual Mean: 50
Estimated Mean: 49.82

```

```

Actual Variance: 16
Estimated Variance: 15.75

```

The estimated values are very close to the true parameters because MLE converges with sufficient data.

3. Building Machine Learning Algorithms

Build a complete classification model.

Answer:

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# Load Dataset
iris = load_iris()

X = iris.data
y = iris.target

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Model
model = LogisticRegression()

model.fit(X_train, y_train)

# Prediction

```

```

y_pred = model.predict(X_test)

# Evaluation
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("Accuracy:", accuracy)
print("\nConfusion Matrix:\n")
print(cm)

```

Example Output

Accuracy: 1.0

Confusion Matrix:

```

[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

```

Analysis

- **Data Preprocessing:** Standardization using `StandardScaler`.
- **Training:** Logistic Regression is trained on the training dataset.
- **Testing:** Predictions are made on the test dataset.
- **Accuracy:** Measures the percentage of correctly classified samples.
- **Confusion Matrix:** Shows the number of correct and incorrect predictions for each class.

4. Neural Networks

Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer:

```

from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Generate non-linear dataset
X, y = make_moons(n_samples=300, noise=0.2, random_state=42)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Neural Network with one hidden layer
model = MLPClassifier(hidden_layer_sizes=(5,),
                      activation='relu',
                      max_iter=1000,
                      random_state=42)

model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

# Plot loss curve
plt.plot(model.loss_curve_)

```

```
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("Neural Network Learning Curve")
plt.show()
```

Analysis

- The hidden layer learns complex (non-linear) relationships.
- ReLU activation helps capture curved decision boundaries.
- The loss decreases with iterations, indicating successful learning.
- The neural network performs much better than linear models on non-linear datasets.

4. 5. Artificial Neurons

Implement a single artificial neuron using Sigmoid and ReLU activation functions.

Answer:

```
import numpy as np

# Input and weights
X = np.array([2, 3])
W = np.array([0.5, 0.8])
b = 1

# Weighted Sum
z = np.dot(X, W) + b

# Sigmoid
sigmoid = 1 / (1 + np.exp(-z))

# ReLU
relu = max(0, z)

print("Weighted Sum:", z)
print("Sigmoid Output:", sigmoid)
print("ReLU Output:", relu)
```

Example Output

```
Weighted Sum: 4.4
Sigmoid Output: 0.9879
ReLU Output: 4.4
```

Comparison

Activation Function	Output	Characteristics
Sigmoid	Between 0 and 1	Smooth, used for probability estimation but may suffer from vanishing gradients.
ReLU	0 or positive value	Faster training, reduces vanishing gradients, widely used in deep learning.

5. 6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets.

Answer:

```
from sklearn.datasets import make_classification, make_circles
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Linearly Separable Dataset
X1, y1 = make_classification(
    n_samples=200,
    n_features=2,
    n_redundant=0,
    n_clusters_per_class=1,
    random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X1, y1, test_size=0.2, random_state=42)

model = Perceptron()

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Linear Dataset Accuracy:", accuracy_score(y_test, pred))

# Non-Linearly Separable Dataset
X2, y2 = make_circles(
    n_samples=200,
    noise=0.1,
    factor=0.4,
    random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X2, y2, test_size=0.2, random_state=42)

model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Non-Linear Dataset Accuracy:", accuracy_score(y_test, pred))
```

Analysis

- The Perceptron performs well on **linearly separable** data.
- It performs poorly on **non-linearly separable** data because it can only learn a linear decision boundary.

b) Train a Multilayer Perceptron (MLP) model and compare its performance with a single-layer perceptron.

Answer:

```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Non-linear dataset
X, y = make_circles(
    n_samples=300,
    noise=0.1,
    factor=0.4,
    random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Perceptron
perceptron = Perceptron(max_iter=1000)
perceptron.fit(X_train, y_train)

pred1 = perceptron.predict(X_test)

# MLP
mlp = MLPClassifier(hidden_layer_sizes=(10,),
                    max_iter=1000,
                    random_state=42)

mlp.fit(X_train, y_train)

pred2 = mlp.predict(X_test)

print("Perceptron Accuracy:",
      accuracy_score(y_test, pred1))

print("MLP Accuracy:",
      accuracy_score(y_test, pred2))
```

Example Output

```
Perceptron Accuracy: 0.52
MLP Accuracy: 0.98
```

. 6. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network and verify using code.

Manual Calculation

Given:

- Inputs: $x_1 = 1, x_2 = 2$
- Weights:
 - $w_1 = 0.5, w_2 = 0.4$
- Bias: $b = 0.1$

Weighted sum:

$$z = (1 \times 0.5) + (2 \times 0.4) + 0.1 = 1.4$$

Sigmoid output:

$$y = \frac{1}{1 + e^{-1.4}} = 0.8022$$

Output ≈ 0.8022

Answer (Python)

```
import numpy as np

# Inputs
X = np.array([1,2])

# Weights
W = np.array([0.5,0.4])

b = 0.1

# Forward Propagation
z = np.dot(X,W) + b
output = 1/(1+np.exp(-z))

print("Weighted Sum =", z)
print("Network Output =", output)
```

Output

```
Weighted Sum = 1.4
Network Output = 0.8022
```

b) Implement backpropagation and update weights after one iteration.

Answer

```
import numpy as np

# Inputs
x = np.array([1,2])

# Initial weights
w = np.array([0.5,0.4])

b = 0.1

target = 1
lr = 0.1

# Forward
z = np.dot(x,w)+b
y = 1/(1+np.exp(-z))

# Error
error = target-y

# Sigmoid derivative
```



```
gradient = error*y*(1-y)

# Weight update
w = w + lr*gradient*x
b = b + lr*gradient

print("Updated Weights:",w)
print("Updated Bias:",b)
```

Analysis

- Forward propagation computes the prediction.
 - Backpropagation calculates the error gradient.
 - Weights and bias are updated to reduce the error.
 - Repeating this process over many iterations trains the network.
-

7. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent and analyze learning rate.

Answer

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-5,5,100)

def cost(x):
    return x**2

learning_rate = 0.1
theta = 4

history=[]

for i in range(50):
    grad = 2*theta
    theta = theta-learning_rate*grad
    history.append(cost(theta))

plt.plot(history)
plt.xlabel("Iterations")
plt.ylabel("Cost")
plt.title("Gradient Descent")
plt.show()

print("Final Theta:",theta)
```

Analysis

- **Small learning rate (0.001):** Very slow convergence.
 - **Medium learning rate (0.1):** Fast and stable convergence.
 - **Large learning rate (1.0):** May overshoot and fail to converge.
-

b) Implement SGD for regression and compare with Batch Gradient Descent.

Answer

```
from sklearn.linear_model import SGDRegressor, LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=500,
                      n_features=1,
                      noise=10,
                      random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Batch Gradient Descent
lr = LinearRegression()
lr.fit(X_train, y_train)

# SGD
sgd = SGDRegressor(max_iter=1000,
                   learning_rate='constant',
                   eta0=0.01)

sgd.fit(X_train, y_train)

print("Batch Score:", lr.score(X_test, y_test))
print("SGD Score:", sgd.score(X_test, y_test))
```

Comparison

Batch Gradient Descent	SGD
Stable convergence	Noisy convergence
Slower	Faster
Uses whole dataset	Uses one sample at a time
High computation	Low computation

. 8. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent.

Answer

```
from sklearn.neural_network import MLPRegressor
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

X, y = make_regression(n_samples=1000,
                      n_features=10,
                      noise=5,
                      random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = MLPRegressor(hidden_layer_sizes=(20,),
                    batch_size=32,
```

```

        max_iter=500,
        random_state=42)

model.fit(X_train,y_train)

print("Test Score:",model.score(X_test,y_test))

```

Analysis

Batch Size	Performance
1 (SGD)	Fast updates but noisy
16–64	Stable and fast convergence
128–512	Smoother updates but slower
Full Batch	Most stable but computationally expensive

Conclusion: Mini-batches of **32 or 64** generally provide the best balance between speed and stability.

. 9. Curse of Dimensionality

Create datasets with increasing dimensions and analyze distance and accuracy.

Answer

```

import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

dimensions = [2,5,10,20,50]

for d in dimensions:

    X,y = make_classification(
        n_samples=1000,
        n_features=d,
        n_informative=min(2,d),
        random_state=42)

    X_train,X_test,y_train,y_test = train_test_split(
        X,y,test_size=0.2,random_state=42)

    model = KNeighborsClassifier()

    model.fit(X_train,y_train)

    pred = model.predict(X_test)

    acc = accuracy_score(y_test,pred)

    distance = np.mean(np.linalg.norm(X_train[:100]-X_train[0],axis=1))

    print("Dimensions:",d)
    print("Average Distance:",distance)
    print("Accuracy:",acc)
    print()

```

Analysis

- As the number of dimensions increases, data becomes more sparse.
- The distance between points becomes more similar, making nearest-neighbor methods less effective.
- Distance-based algorithms such as KNN typically experience reduced accuracy as dimensionality grows.
- Dimensionality reduction techniques like **PCA**, **LDA**, or feature selection can improve performance by reducing irrelevant features.